

CSE4502 (Operating Systems Lab): 1A

Jibon Naher¹ and Samnun Azfar²

¹Assistant Professor, CSE

²Junior Lecturer, CSE

February 3, 2026

Introduction

A **Deadlock** is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource held by some other process. This lab explores how deadlocks occur in a multi-threaded environment and how they can be avoided using mathematical models like the Banker's Algorithm.

The Four Necessary Conditions

For a deadlock to occur, the following four conditions must hold simultaneously:

1. **Mutual Exclusion:** At least one resource must be held in a non-sharable mode; only one process can use the resource at any given time.
2. **Hold and Wait:** A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.
3. **No Preemption:** Resources cannot be preempted; they can only be released voluntarily by the process holding them.
4. **Circular Wait:** A closed chain of processes exists such that each process holds at least one resource needed by the next process in the chain.

Task 1: Simulating a Deadlock in C

Objective: Write a C program using POSIX threads (pthreads) and mutexes to create a circular wait condition.

deadlock_demo.c

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
6 pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
7
8 void* thread1_routine(void* arg) {
9     printf("Thread 1: Trying to lock Mutex 1...\n");
10    pthread_mutex_lock(&lock1);
11    printf("Thread 1: Locked Mutex 1. Sleeping...\n");
12    sleep(1);
13}
```

```

14     printf("Thread 1: Trying to lock Mutex 2...\n");
15     pthread_mutex_lock(&lock2);
16     printf("Thread 1: Locked Mutex 2.\n");
17
18     pthread_mutex_unlock(&lock2);
19     pthread_mutex_unlock(&lock1);
20     return NULL;
21 }
22
23 void* thread2_routine(void* arg) {
24     printf("Thread 2: Trying to lock Mutex 2...\n");
25     pthread_mutex_lock(&lock2);
26     printf("Thread 2: Locked Mutex 2. Sleeping...\n");
27     sleep(1);
28
29     printf("Thread 2: Trying to lock Mutex 1...\n");
30     pthread_mutex_lock(&lock1);
31     printf("Thread 2: Locked Mutex 1.\n");
32
33     pthread_mutex_unlock(&lock1);
34     pthread_mutex_unlock(&lock2);
35     return NULL;
36 }
37
38 int main() {
39     pthread_t t1, t2;
40     pthread_create(&t1, NULL, thread1_routine, NULL);
41     pthread_create(&t2, NULL, thread2_routine, NULL);
42     pthread_join(t1, NULL);
43     pthread_join(t2, NULL);
44     return 0;
45 }

```

Instructions for Task 1

- Compile the code: `gcc deadlock_demo.c -o deadlock -pthread`
- Run the program: `./deadlock`, and check what happens
- Modify the code so that deadlock can be avoided by attacking the Hold and Wait condition.

Task 2: Deadlock Avoidance (Banker's Algorithm)

Objective: Implement the Safety Algorithm part of the Banker's Algorithm to determine if a system is in a safe state.

Example 1: Safe State Scenario

Given 5 processes (P_0 through P_4) and 3 resource types (A, B, C) with total instances (10, 5, 7).

Process	Allocation			Max Demand			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	3	3	2
P_1	2	0	0	3	2	2			
P_2	3	0	2	9	0	2			
P_3	2	1	1	2	2	2			
P_4	0	0	2	4	3	3			

Example 2: Unsafe State Scenario

Modify the available resources in your code to **Available = (0, 2, 0)**.

Helper Code: Banker's Safety Algorithm

bankers_safety.c

```
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 #define N 5 // Processes
5 #define M 3 // Resources
6
7 bool isSafe(int alloc[N][M], int max[N][M], int avail[M], int safeSeq[N]) {
8     // 1. Initialize 'need[N][M]' matrix (Max - Allocation)
9     // 2. Initialize 'finish[N]' boolean array to all false
10    // 3. Initialize 'work[M]' array with 'avail' values
11    // 4. While there are processes to finish:
12    //     a. Find an index 'i' such that finish[i] == false AND need[i] <=
13       work
14    //     b. If no such 'i' exists, return false (System is Unsafe)
15    //     c. If found, add alloc[i] to work, set finish[i] = true, and
16       record 'i' in safeSeq
17    // 5. If all processes are finished, return true (System is Safe)
18
19    /* STUDENT IMPLEMENTATION HERE */
20    return false;
21 }
22
23 int main() {
24     int alloc[N][M] = { {0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0,
25       2} };
26     int max[N][M] = { {7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}
27       };
28     int avail[M] = {3, 3, 2};
29     int safeSeq[N];
30
31     if (isSafe(alloc, max, avail, safeSeq)) {
```

```

28     printf("System is in SAFE state.\nSafe Sequence: ");
29     for (int i = 0; i < N; i++) printf("P%d%s", safeSeq[i], (i == N-1 ?
        "\n" : " -> "));
30 } else {
31     printf("System is in UNSAFE state (Deadlock potential).\n");
32 }
33
34 return 0;
35 }

```

Instructions for Task 2

- Fill the implementation for the `isSafe()` function based on the provided instructions.